



# Introduzione ai Servizi Scalabili e Affidabili (SRS)

Questo modulo definisce le fondamenta dei sistemi distribuiti moderni, analizzando perché la scalabilità e l'affidabilità siano diventate sfide ingegneristiche critiche per le aziende tecnologiche.

## Indice

- [Introduzione ai Servizi Scalabili e Affidabili \(SRS\)](#)
  - [Indice](#)
  - [1. Il Contesto: Perché SRS?](#)
  - [2. Definizione di Scalabilità](#)
    - [Dimensioni della Scalabilità](#)
    - [Strategie di Scaling](#)
  - [3. Definizione di Affidabilità \(Reliability\)](#)
  - [4. Disponibilità \(Availability\) e SLA](#)
  - [5. Il Modello a Microservizi](#)

---

## 1. Il Contesto: Perché SRS?

Negli ultimi decenni siamo passati da applicazioni eseguite su singole macchine a servizi globali che devono servire milioni di utenti simultaneamente. Questo ha reso obsoleta l'idea di "server unico", introducendo la necessità di sistemi che possano crescere e resistere ai guasti in modo automatico.

- **Evoluzione dei Sistemi:** Dai mainframe isolati ai cluster di migliaia di nodi nel cloud.
  - **Complessità Distribuita:** Un sistema distribuito è un insieme di computer indipendenti che appare ai suoi utenti come un unico sistema coerente.
  - **Sfide Principali:** Latenza di rete, guasti parziali (una macchina cade ma le altre continuano a girare) e consistenza dei dati.
-

---

## 2. Definizione di Scalabilità

La scalabilità non è una proprietà "on/off", ma la capacità di un sistema di gestire un aumento del carico aggiungendo risorse.

### Dimensioni della Scalabilità

- **Scalabilità di Carico (Load):** Capacità di gestire più utenti o transazioni senza degradare le performance.
- **Scalabilità Geografica:** Capacità di servire utenti in diverse parti del mondo con bassa latenza.
- **Scalabilità Amministrativa:** Facilità di gestire un sistema che passa da 10 a 10.000 nodi.

### Strategie di Scaling

- **Vertical Scaling (Scaling Up):** Aggiungere più potenza (CPU, RAM) a una singola macchina. Ha limiti fisici e costi esponenziali.
- **Horizontal Scaling (Scaling Out):** Aggiungere più macchine al sistema. È la strategia preferita nel cloud perché potenzialmente illimitata, ma introduce complessità nel software.

[Image of vertical vs horizontal scaling]

---

## 3. Definizione di Affidabilità (Reliability)

Un sistema è affidabile quando continua a svolgere la sua funzione prevista, con il livello di prestazioni richiesto, anche in presenza di guasti.

- **Guasto (Fault) vs Fallimento (Failure):** Un "fault" è un componente che smette di funzionare; un "failure" è l'intero sistema che smette di rispondere. L'obiettivo dell'SRS è creare sistemi **Fault-Tolerant**, dove i singoli guasti non causano il fallimento totale.
- **Tipi di Guasti:**
  - **Hardware:** Rottura di dischi, interruzioni di corrente.

- **Software:** Bug, leak di memoria, eccezioni non gestite.
  - **Umani:** Errori di configurazione (la causa principale della maggior parte dei downtime).
- 

## 4. Disponibilità (Availability) e SLA

L'affidabilità viene spesso misurata tramite la **Disponibilità**, ovvero la percentuale di tempo in cui il sistema è operativo.

- **I "Nove":** Si parla spesso di "disponibilità al 99.9%" (tre nove) o "99.999%" (cinque nove). Passare da tre a cinque nove riduce il downtime annuo da quasi 9 ore a soli 5 minuti.
  - **SLA (Service Level Agreement):** Contratti formali che definiscono il livello di servizio atteso. Se il sistema scende sotto la soglia di disponibilità promessa, il fornitore spesso deve rimborsare il cliente.
- 

## 5. Il Modello a Microservizi

Per ottenere scalabilità e affidabilità, l'industria è passata dall'architettura monolitica ai microservizi.

- **Monolite:** Un unico grande blocco di codice. Se una parte fallisce, cade tutto. Difficile da scalare se non interamente.
- **Microservizi:** Piccoli servizi indipendenti comunicanti via rete. Permettono di scalare solo le parti del sistema sotto carico e isolano i guasti (se il servizio "pagamenti" cade, il servizio "catalogo" può continuare a funzionare).

[Image of monolithic vs microservices architecture]